Document de conception · v1

Kabyle LLM Benchmark

Mesurer la maîtrise du kabyle (taqbaylit) par les LLM, améliorer un petit modèle open source et prouver le gain avec des chiffres.

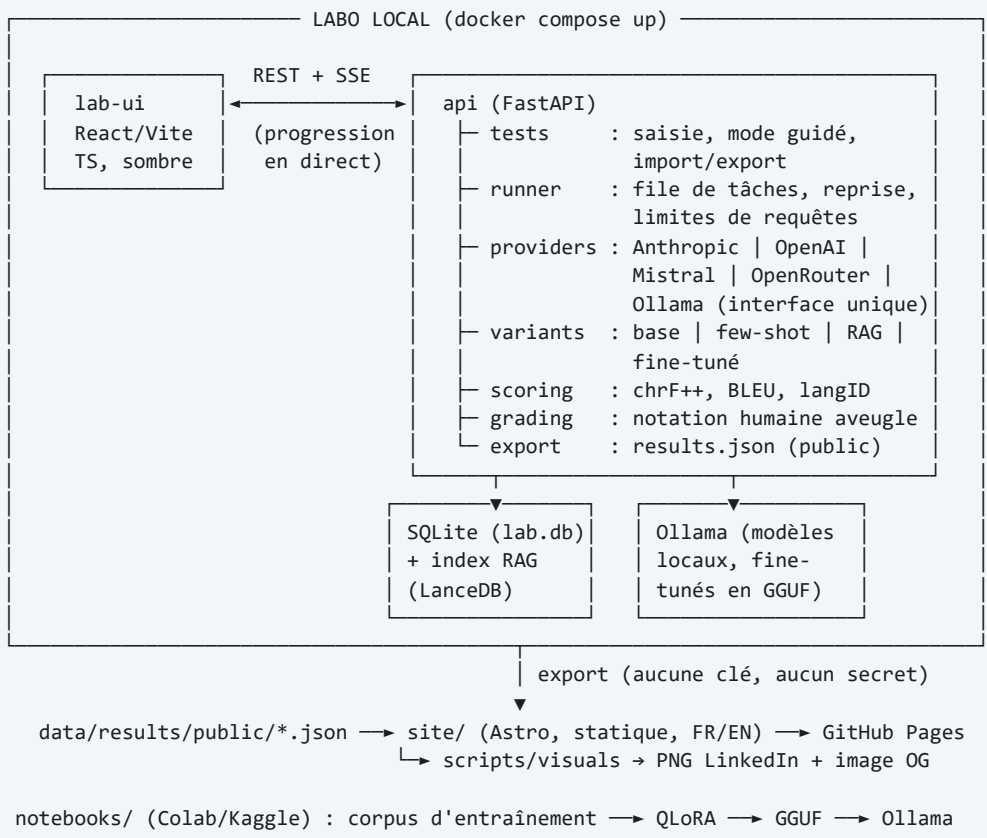
Contenu	Architecture globale, arborescence, format du jeu de tests, métriques, anti-contamination, sécurité, plan d'étapes, décisions validées
Dépôt	github.com/hasdaouiryma16-cmd/kabyle-llm-benchmark
Emplacement local	C:\Users\ADMIN\Projets\kabyle-llm-benchmark
Date	28 octobre 2026
Statut	Architecture validée · étape 0 (squelette du dépôt) réalisée

Code sous Apache-2.0 · données sous CC BY 4.0. Aucun texte en kabyle de ce document n'est généré par IA : les champs kabyles sont volontairement laissés en espaces réservés.

1. Règles fondamentales du projet

#	Règle	Mise en œuvre
1	Toutes les références en kabyle viennent de l'autrice	L'IA ne propose que des consignes en français, des catégories et la structure. Aucun champ kabyle n'est pré-rempli.
2	Aucun résultat inventé	Chaque score est relié à une réponse brute stockée et à une notation réelle ; les résultats publics sont exportés par le labo, jamais écrits à la main.
3	Zéro contamination	Séparation physique test / entraînement, dédoublonnage, contrôle automatique en CI, chaîne canari (section 6).
4	Avancer étape par étape	Validation de l'autrice entre chaque étape ; chaque choix technique est consigné dans `docs/decisions.md`.

2. Architecture globale



Choix techniques et justifications

Choix	Pourquoi

SQLite (SQLModel + Alembic)	Une seule utilisatrice et quelques milliers de lignes : PostgreSQL ajouterait un service sans bénéfice. Un fichier unique, facile à sauvegarder ; migration possible plus tard.
Jeu de tests en JSONL dans Git	Source de vérité versionnée, lisible en diff et citable. La base l'importe et l'exporte.
SSE pour la progression	Flux unidirectionnel serveur → client : plus simple que WebSocket.
Runner idempotent	Chaque appel est identifié par (run, test, modèle, variante, essai). Une relance ne rejoue que ce qui manque ou a échoué. Backoff exponentiel avec aléa, plafond de concurrence par fournisseur, plafond de coût par run.
Interface unique de fournisseurs	Ajouter un modèle = une classe `Provider` + une entrée dans `models.yaml`. OpenRouter sert de repli universel.
Variantes = modèles comme les autres	« Gemma + RAG » est une ligne du classement comme « Claude » : les comparaisons avant/après sont automatiques.
Mêmes paramètres pour tous	Température basse (0 à 0,2), même consigne système, même limite de tokens ; tout est enregistré avec chaque réponse.
Astro pour le site public	HTML statique rapide, balises Open Graph soignées, îlots React pour les graphiques interactifs, i18n FR/EN natif.
LanceDB pour le RAG	Base vectorielle sous forme de fichier, sans serveur, cohérente avec SQLite.

3. Arborescence du dépôt

```
kabyle-llm-benchmark/
├── README.md / README.en.md
├── LICENSE                      # code : Apache-2.0
├── LICENSE-DATA.md              # données : CC BY 4.0
├── CONTRIBUTING.md  CITATION.cff
├── .env.example  .gitignore  .gitattributes  .editorconfig  .pre-commit-config.yaml
├── docker-compose.yml           # api + lab-ui (+ ollama optionnel)
├── Makefile                     # make lab | test | export | site
├── .github/workflows/
│   ├── ci.yml                   # lint, tests, build, gitleaks, contrôle de contamination
│   └── pages.yml                # build + déploiement GitHub Pages
├── backend/
│   ├── pyproject.toml           # uv, ruff, mypy, pytest
│   ├── app/
│   │   ├── main.py  config.py  db.py
│   │   ├── models/              # tables SQLModel
│   │   ├── api/                 # tests, runs, grading, results, export
│   │   ├── providers/           # base, anthropic, openai, mistral, openrouter, ollama
│   │   ├── variants/            # base, fewshot, rag
│   │   ├── runner/              # file de tâches, reprise, rate limit
│   │   ├── scoring/             # chrf, langid, normalize
│   │   └── export/
│   ├── config/models.yaml
│   └── tests/
├── lab-ui/                      # React + TS + Vite (Tailwind, shadcn/ui, Recharts)
│   └── src/pages/               # Tests, Guided, Run, Grade, Dashboard
├── data/
│   ├── test/                    # 🔒 JEU DE TEST — jamais utilisé pour l'entraînement
│   │   ├── kab-bench.v1.jsonl
│   │   ├── private/             # ~20 % non publiés (ignoré par Git)
│   │   ├── MANIFEST.json        # version + sha256 + nb items + canari
│   │   └── DATASET_CARD.md
│   ├── prompts/guided_fr.yaml   # ~50 consignes en français
│   ├── train/                   # 🪄 données d'amélioration
│   │   ├── raw/ (ignoré)  processed/
│   │   └── SOURCES.md           # source, URL, licence, date
│   └── results/public/          # résultats exportés → site
├── pipelines/                   # téléchargement, nettoyage, dédoublonnage, contamination
├── notebooks/                   # 01_qlora_colab.ipynb, 02_merge_to_gguf.ipynb
├── site/                        # Astro FR/EN
├── scripts/visuals/             # PNG 1080×1080 et 1200×627, image OG
└── docs/                        # methodology, metrics, architecture, add-a-model, decisions
```

4. Format du jeu de tests

JSONL (une ligne par test), UTF-8, **normalisé NFC** : le kabyle mélange caractères précomposés et diacritiques combinants (« ḥ » peut s'écrire de deux façons) ; la normalisation évite que deux chaînes identiques à l'œil soient comparées comme différentes.

```
{
  "id": "tr-fr-kab-0001",
  "category": "translation_fr_kab",
  "subcategory": "famille",
  "script": "latin",
  "prompt": "Traduis en kabyle (alphabet latin) : « Ma grand-mère prépare le couscous. »",
  "source_text": "Ma grand-mère prépare le couscous.",
  "context": null,
  "references": ["<ta référence 1>", "<variante acceptable 2>"],
  "grading": "auto+human",
  "expected_lang": "kab",
  "difficulty": 1,
  "source": "author",
  "author": "hasdaouiryma16-cmd",
  "notes": "",
  "created_at": "2026-10-28",
  "updated_at": "2026-10-28"
}
```


Champ	Rôle
`category`	translation_fr_kab · translation_kab_fr · comprehension · culture · writing · conversation
`script`	latin ou tifinagh (catégorie Écriture et métriques)
`context`	Court texte kabyle des tests de compréhension (écrit par l'autrice)
`references`	Plusieurs références possibles : chrF++ plus juste pour une langue à l'orthographe variable
`grading`	auto · human · auto+human
`expected_lang`	Langue attendue en sortie, pour la détection automatique
`difficulty`	1 à 3, pour analyser les résultats par niveau

> Chaque run enregistre `test_set_version` et l'empreinte `sha256` du jeu : un score n'est jamais comparé entre deux versions différentes.

Catégories

Traduction fr → kab

Phrases du quotidien, famille, nature, émotions, nombres, temps.

Traduction kab → fr

Le modèle comprend-il le kabyle écrit ?

Compréhension

Court texte kabyle + question.

Culture

Proverbes, expressions, coutumes.

Écriture

Alphabet latin (ɛ, ḥ, ṭ, ẓ, ɣ, č, ǧ, ṛ, ṣ, ḍ) et tifinagh.

Conversation

Le modèle répond-il en kabyle, et non en arabe ou en français ?

5. Notation

Automatique

- **chrF++ (principale)** — n-grammes de caractères : robuste à la morphologie riche et à l'orthographe variable. Standard pour les langues peu dotées (FLORES, AmericasNLP).
- **BLEU (secondaire)** — pour la comparabilité avec la littérature, via sacreBLEU avec signature publiée.
- **Pas de COMET** — repose sur XLM-R, qui couvre mal le kabyle : scores trompeurs. Limite documentée.
- **Détection de langue : GlotLID** (fastText, 1 600+ langues dont `kab_Latn`) + détection d'écriture par plages Unicode (déterministe). GlotLID est un indicateur, pas une vérité, surtout sur textes courts.

Humaine, à l'aveugle

Note	Signification (grille à affiner avec l'autrice)
0	Faux, hors sujet, ou mauvaise langue
1	Partiellement correct, erreurs importantes
2	Correct sur le fond, erreurs mineures (orthographe, tournure)
3	Correct et naturel

Les réponses sont mélangées et anonymisées (le modèle n'est révélé qu'après), avec raccourcis clavier 0–3. La note humaine fait foi ; la corrélation entre métriques automatiques et notes humaines est publiée.

6. Anti-contamination : séparation vérifiable

1. **Séparation physique** : `data/test/` vs `data/train/`. Les modules few-shot et RAG ne lisent que `data/train/processed/` (chemin imposé et testé).
2. **Dédoublonnage** : normalisation (NFC, minuscules, ponctuation) puis suppression des doublons exacts et quasi-doublons (chrF / MinHash au-dessus d'un seuil) face à toute chaîne du test (référence, source, contexte).
3. **Contrôle en CI** : `check_contamination.py` fait échouer le build si une phrase du test apparaît dans un corpus, un exemple few-shot ou l'index RAG.
4. **Chaîne canari** : un GUID unique intégré au jeu publié (comme BIG-bench) permet de détecter un futur entraînement sur nos tests.
5. **Partie privée (~20 %)** : jamais publiée, elle reste une mesure propre même si le jeu public finit dans les données d'entraînement de futurs modèles.

7. Sécurité des secrets

Niveau	Protection

Fichiers	`.env` ignoré par Git dès le premier commit ; `.env.example` sans valeurs.
Avant commit	Hook pre-commit : gitleaks + blocage de tout fichier `.env*` (sauf l'exemple) + refus des gros fichiers.
CI	gitleaks sur tout l'historique + vérification qu'aucun `.env` n'est suivi.
Export public	Liste blanche de champs : jamais de configuration ni de clé.
Réseau	Le labo n'écoute que sur 127.0.0.1 et n'est jamais déployé.

8. Phase 2 : pistes de petits modèles (à confirmer)

Choix définitif au début de la phase 2, après vérification des modèles disponibles à ce moment-là.

Modèle	Atouts	Points d'attention
Gemma 3 4B	Très multilingue (140+ langues), bon rapport taille/qualité	Licence Gemma (usage libre avec conditions)
Qwen3 4B	Licence Apache-2.0, excellent support du fine-tuning	Plutôt orienté chinois/anglais
Llama 3.2 3B	Écosystème très large, nombreux tutoriels QLoRA	Licence Llama Community, multilinguisme limité

Trois approches, de la plus simple à la plus avancée, chacune ajoutée comme **variante** comparable : few-shot → RAG (embeddings multilingues) → QLoRA (Colab / Kaggle, export GGUF servi par Ollama). Coûts et durées estimés présentés **avant** tout lancement.

9. Plan par étapes

#	Étape	Statut
0	Squelette du dépôt, .gitignore, licences, protections anti-secrets, premier commit	**Fait**
1	Backend : modèle de données, API des tests, import/export JSON/CSV, manifeste	À venir
2	UI de saisie, mode guidé (~50 consignes FR), barre de caractères spéciaux, mode sombre	À venir
3	Fournisseurs + runner (mode test peu coûteux d'abord)	À venir
4	Notation automatique + notation humaine à l'aveugle	À venir
5	Tableau de bord → premiers résultats réels (fin phase 1)	À venir
6	Phase 2 : sources et licences, pipeline, few-shot, RAG, notebooks QLoRA	À venir
7	Phase 3 : comparaisons avant/après et analyse des erreurs	À venir
8	Site public FR/EN, visuels LinkedIn, documentation finale	À venir

10. Décisions validées

Sujet	Décision
Base de données	SQLite (SQLModel + Alembic)
Licence du code	Apache-2.0

Licence des données	CC BY 4.0
Jeu privé	Oui, ~20 % non publiés
Prérequis à installer	Python 3.12 (via uv), Node.js 22 LTS, Ollama — Git et Docker déjà présents

11. Publier le dépôt sur GitHub

1. Sur github.com, se connecter puis **New repository** → nom `kabyle-llm-benchmark`, visibilité Public, **sans** README, .gitignore ni licence (ils existent déjà localement).
2. Dans PowerShell :

```
cd C:\Users\ADMIN\Projets\kabyle-llm-benchmark
git remote add origin https://github.com/hasdaouiryma16-cmd/kabyle-llm-benchmark.git
git push -u origin main
```

3. À la première connexion, Git Credential Manager ouvre le navigateur pour l'authentification GitHub.
4. Dans *Settings* → *Code security*, activer **Secret scanning** et **Push protection** (dernière barrière côté GitHub).